

Deep-Dive Explanation — mediapipe_wrist_debug.py

Overview

This ROS 2 node subscribes to a color image, the aligned depth-to-color stream, and the color CameraInfo. It runs MediaPipe Hands to detect up to two hands, draws the landmarks on the color image, samples depth around the wrist landmark (index 0), computes a robust z estimate (meters) and the true range $R = \sqrt{X^2 + Y^2 + Z^2}$ by deprojecting with the camera intrinsics, overlays these values on the image, and publishes the result to /wrist/debug_image.

Parameters & QoS

Declared ROS 2 parameters (with defaults):

- color_topic: /camera/cam2/color/image_raw
- depth_topic: /camera/cam2/aligned_depth_to_color/image_raw
- info_topic: /camera/cam2/color/camera_info

Subscriptions use QoSProfile.SENSOR_DATA (BEST_EFFORT, KEEP_LAST), which matches typical camera publishers and avoids RELIABLE/BEST_EFFORT mismatches. Published debug images use queue depth 10.

Node state & libraries

Uses CvBridge for ROS↔OpenCV conversions and image_geometry.PinholeCameraModel to read fx, fy, cx, cy from CameraInfo. Keeps the latest depth frame (in meters) and a flag have_info once intrinsics are loaded. Also records depth_frame_id for an alignment sanity check.

MediaPipe configuration

mp.solutions.hands.Hands with:

- static_image_mode = False (tracking after initial detection)
- max_num_hands = 2
- min_detection_confidence = 0.3
- min_tracking_confidence = 0.3

Drawing utils render landmarks and connections on the BGR image.

Subscriptions & callbacks

1) CameraInfo → on_info:

- Initializes the pinhole camera model, logs intrinsics and frame_id, and sets have_info=True.

2) Depth → on_depth:

- Converts depth with CvBridge using passthrough.
- If uint16, interprets as millimeters and converts to meters.
- If float32, assumes meters already.
- Other types trigger a warning and skip updates.
- Stores depth_frame_id from the message header.

3) Color → on_color:

- Converts to BGR8, caches h, w, converts BGR→RGB, and runs MediaPipe Hands.

- Extracts handedness labels ("Left"/"Right") if available.

- For each detected hand:

- Draws landmarks and connections.

- Gets wrist landmark (index 0), converts normalized [0,1] coords to pixel (u, v) with clipping.

- If depth or intrinsics are missing, still draws/labels the wrist pixel (no numbers shown).

- If available:

- Extracts a 7×7 patch around (u, v), keeps non-zero values, takes the median as z (meters).

- Alignment check: if depth_frame_id differs from the expected color optical frame, logs a warning to enable alignment.

- Deprojection: $X = (u - cx) * z / fx$, $Y = (v - cy) * z / fy$, $Z = z$; $R = \sqrt{X^2 + Y^2 + Z^2}$.

- Overlays text: z=...m | R=...m near the wrist.

- Publishes the annotated frame to `/wrist/debug_image` every color callback.

Robustness choices

- Patch median (7×7) rejects outliers/holes common in hand regions.
- Graceful degradation: if depth/intrinsics aren't ready, the node still shows landmarks and pixel coordinates.
- Alignment warning nudges you to enable depth-to-color alignment when frames mismatch.

Quick customization

- Pass real topics via `--ros-args -p color_topic:=... -p depth_topic:=... -p info_topic:=...`
- Tweak MediaPipe thresholds for your lighting/scene.
- Increase the window size if depth is especially noisy (e.g., 9×9).

Limitations

- CPU MediaPipe; performance depends on input resolution.
- Only the wrist's z and R are shown; no TF/3D publish (debug node).
- Assumes depth is aligned to color and CameraInfo describes the color stream used for detection.